University of Glasgow

Monday 26 April 2021, Available from 14:00 BST, Expected Duration: 2 hours, Time Allowed: 4 hours, Timed exam within 24 hours

—

!!!!!!!!!!!!!!!! SOLUTIONS !!!!!!!!!!!!!!!!”

DEGREE of MSc

INTRODUCTION TO DATA SCIENCE AND SYSTEMS (M)

Answer all 3 questions

This examination paper is an open book, online assessment and is worth a total of 60 marks.

1. Computational linear algebra and optimisation

You have been asked to help design the subcomponents of a music streaming service. The service has access to 101,750 music tracks (i.e. the audio files). Each music track can be summarised based on the audio content using so-called audio features resulting in a 15 dimensional vector, $\mathbf{x} \in \mathbb{R}^{1x15}$, for each track. The meaning and importance of the individual dimensions in the vector is unknown. The vectors for the individual tracks are collected in a matrix X as row vectors.

Aside from the audio file itself, the service has access to the title and artist for each track, the genre(s) associated with each track (e.g. jazz) and finally the popularity of each track as a scalar $y \in \mathbb{R}$.

(a) The team wants to develop a function called "What is this track called?" where users can upload an audio file with the purpose to identify the name of the track and artist. To this end we are interested in computing Euclidian distances between the music tracks based on their vector representations.

(i) Certain aspects of X is summarised in Table 1. Explain why it is a good idea to normalise the data in X before computing the similarity between the tracks and suggest a suitable normalisation approach. Justify your approach and make reference to specific elements in Table 1. [3]

Solution:
- Two of the dimensions appear to be on a difference scale (mean) (3 & 8) and two with sig. different variance (1 & 12) than the rest. This will influence the computing of the L2 norm. Given that we don't know the meaning of the dimensions, we have no indication that certain are more important than others and we therefore need to normalise the data to ensure we are not biasing the distance computation [2].
- Normalisation: Remove the mean [0.5] and ensure equal variance [0.5] (or do a full whitening).

(ii) Design a simple search routine which can find the closest match between the uploaded track and a track in the existing dataset. Write the procedure using equations or NumPy code (1-3 lines). Determine how many individual distances you will need to compute and discuss any potential scaleability issues. [3]

Solution:
- Find the best index $i^* = argmin_i\{\, ||\mathbf{x}^* - \mathbf{x}_i||_2^2 \,|\, i = 1,..,N\}$; look up artist and track name with index i^* and return. Note: a concise and correct text description would also be acceptable. [2]
- The procedure requires us to compute the distance to all the elements in the dataset (i.e. 101,750 tracks), i.e. it scales linearly in the number of tracks which can become prohibitively expensive when we have millions of music tracks [1].

Marking: Partial or fulls marks should be awarded to other sensible solutions based on correctness and completeness.

Dim.	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
μ	0.1	-1.5	78.1	0.1	1.1	0.8	0.0	-159.3	0.2	0.0	0.0	0.1	0.0	-0.5	0.3
σ^2	0.01	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	10.1	1.0	1.0	1.0
Min	0.09	-4.0	75.1	-2.4	-1.9	-1.5	-2.9	-162.3	-3.7	-2.5	-2.4	-24.7	-2.6	-3.6	-2.6
Max	0.12	1.6	80.5	3.0	3.8	3.2	2.3	-156.9	2.5	2.6	2.3	31.0	2.0	2.1	2.5

Table 1: Basic statistics for each dimension in X.

(b) A subcomponent of the system relies on a mapping from tracks to popularity. This can be formulated as a matrix problem: $X\mathbf{w}^T - \mathbf{y} = \mathbf{0}$, where X is a matrix containing the music features for the tracks. **w** is a 15 dimensional vector and **y** is vector containing the popularity scores for each track. The team is interested in the most efficient and robust method for finding **w** using the squared error as the loss function.

(i) Specify the dimensions of the matrix X and determine if **w** and **y** are considered row or column vectors, respectively. [1]

> **Solution:**
> - X is a 101,750 x 15 matrix
> - **w** is a row vector, 1x15 (15x1 when transposed)
> - **y** is a column vector, 101,750x1
>
> Marking: Partial marks proportional to correctness and completeness.

(ii) Determine a method for solving the matrix equation wrt. **w**. Justify your approach. [2]

> **Solution:**
> - It is a standard least-squares problem [1] and we can use the pseudo-inverse [1].
> - An alternative solution would be to do use numerical optimisation (e.g. gradient decent) however it is not efficient nor necessarily robust (tweaking of hyper-parameters). Typically no mark or partial marks depending on justification.
>
> Marking: Partial marks proportional to correctness and completeness.

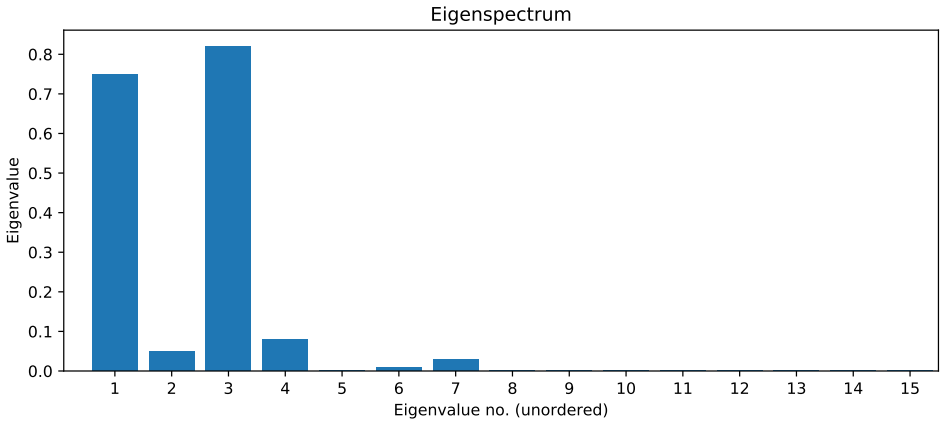

Figure 1: Eigenspectrum (unordered)

(c) The user interface team has requested that you provide a procedure for projecting the music tracks to 2D or 3D based on the vector representation so they can visualise the music tracks on a computer screen. You must use a linear map due to computational constraints.

(i) Outline a procedure for finding the 3D coordinates so the projection preserves most of the variance and can be implemented using only basic Python and NumPy by a junior data scientist. You should not provide the code, but explain the individual steps in the procedure using text or equations and only recommend the suitable NumPy commands. You must specify the dimension of the all vectors or matrices required to compute the projection. [4]

Solution:
- Take the vectors representing the tracks. Collect them in in X (101.750x15), compute the 15x15 covariance matrix (use np.cov)
- Compute the eigenvectors (15x15) of the covariance matrix (use np.eig, np.eigh or np.svd) [1]
- Order the eigenvalues (1x15 or 15x1) (largest to smallest) [1]
- Take the three dominant eigenvalues and their corresponding eigenvectors, collect the vectors in V (15x3) [1]
- Project by applying X to V (use @ or np.matmul), $P = XV$ [1]

(ii) The eigenspectrum of the covariance matrix of X is shown in Figure 1. Discuss what the eigenspectrum says about the vectors representing the audio files and how this could be leveraged to make the system more efficient. Discuss if the team's idea of a 2D or 3D interface is justified. [3]

Solution:
- The covariance matrix is low-rank as many eigenvalues are zero which implies that many of the features are simply linear combinations of other features [1]. We can limit the number of features we need to measure/store [1].
- Most of the variance (well above 90 pct.) is explained by the two principle components (1,3) [1] thus a 2D interface should be enough to capture most of the variance and provides a simpler interface in general)[1]. Alternatively, it is fair to argue for a 3D solution but it would need to be based on careful and sound arguments from the visual/HCI perspective (which is not covered in IDSS).

(d) Your team is contemplating a new subcomponent which would enable users to generate a new music track. The team has already developed a function, $r(\mathbf{x})$, which makes it possible to map from the vector representation, $\mathbf{x}$, to the audio file.

The aim is to create a new track based on a genre profile which is a 5 dimensional vector, $\mathbf{g} \in \mathbb{R}^5$. Your team has provided a **non-linear function**, $f : \mathbf{x} \rightarrow \mathbf{g}$, that maps from the track vector to the 5D genre profile. Provide a solution in the form of an optimisation problem and determine a suitable method to solve the stated problem. Justify your choice of method and explain under which circumstances it is guaranteed to converge to a sensible solution in this scenario. You will need to make assumptions which must be clearly stated. [4]

Solution:

- Optimisation problem:

$$\mathbf{x}^* = \arg\min_{\mathbf{x}} \|\mathbf{g} - f(\mathbf{x})\|_2^2$$

Note: any norm is fine and it is Okay to leave out the square. Important that the students realise that they need to optimise wrt x (and not e.g. parameters of f). [2]

- We cannot assume a closed form solution as f(x) is non-linear (in an unspecified way) - and possibly even unknown so we need to use numerical optimisation. [1]

- If f is **assumed** to be differential with known/computable derivative: gradient decent is the obvious choice. Any random search or stochastic hill-climbing will take a long time due to the relatively high dim. [0.5] Convergence: Assuming Lipschitz continuity (in relation to f) and a suitable step size the optimization will converge to a local optimum. [0.5]

OR

- If f is **assumed** discontinuous or unknown: Stochastic hill-climbing is a viable option [1]. Convergence: With a suitable neighborhood expansion, stochastic methods will converge but very slowly [0.5]. Alternatively, we expect student to list a method from the course, but any zero-th order method is acceptable (e.g. Bayesian optimization, evolutionary. or approx. gradient methods etc). [0.5]

Moreover, we have assumed r(x) is a reasonably accurate function (a deduction of 0.5 mark if this assumption is not stated).

Marking: Open-ended question. Students need to specify their assumptions. Partial marks should be awarded for a sensible solution without assumptions/justifications (default option is to deduct half the marks for that subpart).

2. Probabilities & Bayes rule

Consider a scenario where you are in charge of analysing the data and modelling a pandemic. We consider a given disease (let's call it 'VIRUS'), which has an unknown prevalence ρ in the population (we will assume that $\rho \in [0,1]$ is the proportion of the population that has the disease). We will write the probability that a person is diseased as $p(D) = \rho$.

(a) Your lab has developed a fast testing procedure to detect this disease. In order to evaluate the accuracy and reliability of this test, you have conducted trials on 132 subjects, and compared the results of your test with perfectly accurate (supposedly more expensive) diagnostic. The results of those trials are collated in the following table:

	positive	negative
diseased	28	3
healthy	12	89

(i) Using Bayes formula, and the trial data in the table, provide an estimate of the probabilities:

- $p(D|T)$, that a subject who tested positive is truly diseased; and
- $p(D|\bar{T})$, that a subject who tested negative is actually diseased. [4]

Solution: Bayes formula allows us to formulate the the probability of a true positive as:

$$p(D|T) = p(T|D)p(D)/P(T)$$

[1] we can then estimate the probabilities $p(T|D)$, $p(D)$ and $p(T)$ from the experimental data in the table [1]:

$$\begin{aligned} p(D|T) &= p(T|D)p(D)/P(T) && (1)\\ &= 28/31 \times 31/132/(40/132) && (2)\\ &\simeq 0.9 \times 0.23/0.30 && (3)\\ &= 0.69 && (4) \end{aligned}$$

[1]
the probability of

$$\begin{aligned} p(D|\bar{T}) &= p(\bar{T}|D)p(D)/P(\bar{T}) && (5)\\ &\simeq 3/92 \times \times 31/132/(92/132) && (6)\\ &\simeq 0.03 \times 0.23/0.7 && (7)\\ &\simeq 0.009 && (8) \end{aligned}$$

[1]

(ii) Taking into consideration the test accuracy and reliability as evidenced in the trials,

would this test be appropriate for the following situations:

1. regular testing of people working with vulnerable populations;
2. deciding on whether to administer a treatment with severe side effects; or
3. applying to the whole population to find all diseased individuals

(justify your answers).

[3]

Solution: The test will miss few cases of infected people, but return a significant number of false positives (about 1 in 3!). Therefore it is NOT appropriate for recommending treatment with severe negative side effects (2), [1]
or to be applied indiscriminately to the whole population (3). [1]
On the other hand, the test misses only few diseased individuals and therefore could be used for screening people working with vulnerable populations. [1]

(iii) You administer a test with probabilities $p(D|T) = 0.7$ and $p(D|\bar{T}) = 0.01$ to a sample of 1000 subjects drawn randomly from the population. The test returns 980 negatives and 20 positives.
From this data, calculate an estimate of the prevalence $p(D)$ explaining your reasoning.

[4]

Solution: We can write the prevalence as the probability:

$$p(D) = p(D|T)p(T) + p(D|\bar{T})p(\bar{T})$$

[1]
The main aspect in this question is to realise that some positive tests will be returned by healthy subjects ($20 \times (1 - 0.7) = 6$) but also that some infected subjects will be missed by the test ($980 \times 0.01 \simeq 10$). [1]
Putting it all together, we obtain

$$\begin{aligned} p(D) &= p(D|T)p(T) + p(D|\bar{T})p(\bar{T}) && (9)\\ &= (20 \times 0.7 + 980 \times 0.01)/1000 && (10)\\ &\simeq 0.038 && (11) \end{aligned}$$

[1]
therefore we end up with a prevalence of 3.8% in the population [1]

(b) Let us consider that you are experimenting with a vaccine against the disease. You have 1000 subjects in group A who take the vaccine and 1000 in group B who take a placebo. Let us assume that you test the subjects in both groups daily, and after one month you obtain the following results: 2 subjects from group A tested positive at some point during the month, and 40 subjects from group B. In this part we will assume that we are using a test with the following statistics:

- the probability of having the disease if tested positive is $p(D|T) = 0.7$
- the probability of having the disease if tested negative is $p(D|\bar{T}) = 0.01$.

(i) Accounting for the limitations of the test, how many subjects in group A and B did possibly catch the disease during this month? [5]

Solution: We write the probability to be tested positive (T) in the placebo group (B) as

$$p(T|B) = \frac{40}{1000}$$

[0.5]
because of the test reliability, the probability to be diseased is

$$p(D|B) = p(D|T)p(T|B) + p(D|\bar{T})p(\bar{T}|B)$$

We have $p(D|T) = 0.7$ and $p(D|\bar{T}) = 0.01$, therefore we get

$$p(D|B) = 0.7 \times 0.04 + 0.01 \times 0.96 = 0.0376$$

In other words we estimate that probably only 37 subjects caught the disease (accounting for the limitations of the test). *([1] for the formulation, [1] for the result)*
Similarly, we have:

$$p(T|A) = \frac{2}{1000} = 0.002$$

[0.5] Hence,

$$p(D|A) = p(D|T)p(T|A) + p(D|\bar{T})p(\bar{T}|A) = 0.7 \times 0.002 + 0.1 \times 0.01 = 0.0024$$

In other words we estimate that between 2 and 3 subjects may have had the disease. *([1] for the formulation, [1] for the result)*

(ii) The efficacy of a vaccine is typically calculated as

$$E_V = \frac{p(D|\bar{V}) - p(D|V)}{p(D|\bar{V})}.$$

Use your results from above to calculate the efficacy of the vaccine. Discuss what would happen if your test were less accurate: What would happen if $p(D|T)$ would be lower? If $p(D|\bar{T})$ would be higher? [4]

Solution: The efficacy of the vaccine is therefore $(0.0376 - 0.0024)/0.0376 = 0.936$ so 93.6% [1]
If the accuracy of the test were to be lower, the estimated vaccine efficacy would be lower (although this would of course not affect the true efficacy).

If $p(D|T)$ is lower, this means that the test will be more likely to be positive for healthy people, therefore the statistics will be calculated on more positive cases overall, and the vaccinated group will appear to have more infections. [1]
If $p(D|\bar{T})$ is higher, then the test will miss more of the diseased subjects and the control group will appear to be less affected. [1]
In both cases, the vaccine will appear to be less effective at suppressing infection. [1]
(taken to the extreme, a random test would lead us to believe that the vaccine has no effect).

3. Database systems

Consider a relation Student(ID, Name, StudyPlan) – abbreviated as S – where the primary key (ID) is a 64-bit integer, the Name attribute is a 40-byte (fixed length) string, and StudyPlan is a 16-bit integer. Further consider a relation Marks(ID, CourseID, AssessmentID, Mark) – abbreviated as M – with ID being a foreign key to Student's ID, CourseID and AssessmentID being 16-bit integers, Mark being a 64-bit float, and the first three attributes making up the relation's (composite) primary key.

Assume that both relations are stored in files on disk organised in 512-byte blocks, with each block having a 10-byte header. Assume that S has $r_S = 1,000$ tuples and that M has $r_M = 100,000$ tuples. Last, assume that Student is stored organised in a heap file, and Marks is stored organised in a sequential file sorted by its primary key. Note that the database system adopts fixed-length records – i.e., each file record corresponds to one tuple of the relation and vice versa.

(a) Compute the blocking factors and the number of blocks required to store these relations. Show your work.

[2]

Solution: (Bookwork for knowledge of the concepts, Problem solving otherwise)
Each record (tuple) of S has size: 8 + 40 + 2 = 50 bytes. Thus, the blocking factor will be:

$$bfr_S = \left\lfloor \frac{block\ size - header\ size}{record\ size} \right\rfloor = \left\lfloor \frac{512-10}{50} \right\rfloor = 10 \text{ records per block.}$$

The file will then contain:

$$n_S = \left\lceil \frac{num\ tuples}{blocking\ factor} \right\rceil = \left\lceil \frac{1000}{10} \right\rceil = 100 \text{ blocks.}$$

Similarly, each record of M has size: 8 + 2 + 2 + 8 = 20 bytes. Thus, the blocking factor will be:

$$bfr_M = \left\lfloor \frac{block\ size - header\ size}{record\ size} \right\rfloor = \left\lfloor \frac{512-10}{20} \right\rfloor = 25 \text{ records per block.}$$

The file will then contain:

$$n_M = \left\lceil \frac{num\ tuples}{blocking\ factor} \right\rceil = \left\lceil \frac{100{,}000}{25} \right\rceil = 4,000 \text{ blocks.}$$

(1 mark for the computation of the blocking factors, 1 mark for the number of file blocks)

(b) Consider the following query:

```
SELECT S.Name, M.ID, M.Mark FROM Student as S, Marks as M
WHERE S.ID = M.ID AND S.ID >= 10,000 and S.ID <= 10,199;
```

Assume that the memory of the database system can accommodate $n_B = 22$ blocks for processing, that all student IDs in the query range exist in the database, and that all students have the same number of marks on average.

Consider the following two approaches: (a) S is scanned at the outer loop (9 marks total)

and (b) M is scanned at the outer loop (9 marks total). For each approach, explain the query processing algorithm (3 marks for each approach) taking care to include the file organisation in your reasoning, and estimate the total expected cost (in number of block accesses) for these two strategies (6 marks for each approach). Disregard the cost associated with the writing of the result set. Show your work.

[18]

Solution: (Bookwork for knowledge of the concepts, Problem solving and Synthesis otherwise)

Let $NDV(a,A)$ be the number of distinct values of attribute a in relation A. Then, $NDV(ID,Student) = 200$ (as only values between 10,000 and 10,199 are considered) and $NDV(ID,Mark) = 200$ (foreign key).

As S is stored in a heap file, we need to scan all of its blocks during join processing($n_S = 100$ blocks). However, as ID is the relation's primary key, only 200 records of S will participate in join processing, hence for the purposes of this query $r'_S = NDV(ID,Student) = 200$.

[**1 mark for realising that not all records of S participate in the join result, 1 mark for n_S, 1 mark for r'_S**]

On the other hand, M is stored in a sequential file sorted by primary key. As ID is the first component of the sort (primary) key, all records of interest will be stored contiguously on disk. Hence, scanning M consists of a binary search to locate the block storing the first record with the lowest ID in the range of interest, followed by a sequential scan until we find the first block storing a record with ID outside the range of interest. For the first, binary search, part, we'd need to read $\lceil log_2(4000) \rceil = 12$ blocks. For the second, sequential scan, part, when M is the inner relation, we need to read $r_{M_{inner}} = r_M / r_S = 100,000/1,000 = 100$ records on average per value of ID, translating to 4 blocks ($bfr_M = 25$) per value of ID, or $n_{M_{inner}} = 16$ blocks. If M is the outer relation, then we'd need again 12 block accesses to get to the first block containing a record with ID=10,000, plus a sequential scan till we find the first block with a record with ID=10,200; that is, $n_{M_{outer}} = NDV(ID,Mark) \times 4 = 200 \times 4 = 800$ blocks, mapping to $r_{M_{outer}} = 20,000$ records, and a grand total of $n'_{M_{outer}} = 812$ blocks.

[**2 marks for realising M shouldn't be fully scanned, 1 mark for $n_{M_{inner}}$, 1 mark for $n_{M_{outer}}$, 1 marks for $n'_{M_{outer}}$, 1 marks for $r_{M_{inner}}$, 1 marks for $r_{M_{outer}}$**]

Note: Students may opt to name the above differently and compute them as part of the discussion of the two approaches below. Full marks for these components should be awarded as long as the students have realised that not all records of S participate in the join processing, that M shouldn't be fully scanned but a binary search should be used instead, and the relevant quantities have been computed using correct formulae.

If the outer loop in the nested-loop join scans over relation S, then we need to:

1. Scan all blocks of S, $n_B - 2$ blocks at a time (the latter not being significant in this case).
2. For each ID value in each such block of S, read all blocks of M with that ID value (one at a time) and produce join results.
3. Store join results in the output block until full; flush and continue.

Then, the cost is:

$$n'_S + r'_S \times n'_{M_{inner}} = 100 + 200 * 16 = 3300 \text{ block accesses.}$$

[**3 marks for algorithm – one per step; 1 mark for computation**]

If the outer loop in the nested-loop join scans over relation M, then we need to:

1. Scan all blocks of M, $n_B - 2$ blocks at a time.
2. For each such block, scan all blocks of S for matches and produce join results.
3. Store join results in the output block until full; flush and continue.

Then, the cost is:

$$n''_{M_{outer}} + \left\lceil \frac{n'_{M_{outer}}}{n_B - 2} \right\rceil \times n'_S = 812 + 40 \times 100 = 4812 \text{ block accesses.}$$

[**3 marks for algorithm – one per step; 1 mark for computation**]

Note: Students may opt to blindly apply the nested loop join equations. In that case, the cost for S being the outer relation would be:

$n_S + \left\lceil \frac{n_S}{n_B - 2} \right\rceil \times n_M = 100 + 5 * 4000 = 20100$ block accesses;

and the cost for M being the outer relation would be:

$n_M + \left\lceil \frac{n_M}{n_B - 2} \right\rceil \times n_S = 4000 + 200 * 100 = 24000$ block accesses. This is an obviously suboptimal solution and should thus receive only partial marks, as discussed above.